

RePlot

Land Readjustment & Replotting Software

Comprehensive Architecture & Implementation Guide

Application Configuration, Logging, Dependency Injection & Middleware

Version 1.0 | March 2026

Chapter 1: What Your Canvas Already Does Right

Before touching any code, it is critical to understand what is already working well. Your RePlot canvas architecture follows several industry patterns that professional CAD and GIS applications use. This chapter explains each pattern so you understand WHY it works, not just THAT it works.

1.1 Separation of Concerns (Your Class Architecture)

Your canvas is split into five distinct classes, each with exactly one job. This is the same architecture pattern used internally by AutoCAD, ETABS, and QGIS.

Class	Single Responsibility	Industry Parallel
DrawingEngine	Coordinate math only: world-to-screen, screen-to-world, zoom, pan calculations	AutoCAD ViewportTableRecord
CanvasRenderer	Pixel-level rendering: grid, axis markers, overlays, snap glyphs	GDI+ rendering pipeline
OptimizedDeferredRenderer	Bitmap cache strategy: cache shapes to off-screen bitmap, blit during pan	AutoCAD display list cache
ShapeManager	Shape storage + spatial queries via NTS R-Tree index	QGIS QgsSpatialIndex
DrawingCanvasControl	Event handling + coordination only. Zero math, zero rendering logic	MVC Controller pattern

Why This Matters

If you ever need to swap rendering engines (say from GDI+ to SkiaSharp), you only change CanvasRenderer. DrawingEngine, ShapeManager, and DrawingCanvasControl remain untouched. This is the power of separation of concerns.

1.2 Rendering Performance (Your Three-Tier Strategy)

Your OptimizedDeferredRenderer implements the exact same tiered rendering approach that professional applications use. Here is what each tier does and why:

Shape Count	Strategy	Quality	Why
< 1,000	AntiAlias + bitmap cache	Maximum quality	Few shapes = fast render. Cache to bitmap for instant redraws
1,000 - 5,000	HighSpeed + bitmap cache	Balanced	Skip anti-aliasing to maintain frame rate while caching
> 5,000	LOD + direct draw, no cache	Fast only	Skip sub-pixel shapes entirely. Cache would be too expensive to rebuild

How Pan Optimization Works (Your Code)

During a mouse drag (pan), your code does NOT re-render all geometry. Instead:

1. `BeginPan()` snapshots the current rendered bitmap into `_panBuffer`
2. `DrawShapesDuringPan()` blits `_panBuffer` shifted by screen-pixel delta using `NearestNeighbor` interpolation
3. `EndPan()` triggers a full `RenderNow()` at the new viewport position

This is exactly what AutoCAD and Google Maps do. The user sees instant feedback during drag, and the high-quality render happens only when they release the mouse.

1.3 Object Pooling in CanvasRenderer

Your `CanvasRenderer` creates `Pens`, `Brushes`, and `Fonts` once in `InitializeResources()` and reuses them across every paint frame. This prevents garbage collection pressure that would cause frame stuttering.

Critical Performance Rule

NEVER create a new `Pen()`, `Brush()`, or `Font()` inside a paint event handler. Each allocation triggers eventual GC which causes visible frame drops. Your code already follows this rule correctly.

1.4 Spatial Index (R-Tree via NetTopologySuite)

Your `ShapeManager` uses NTS `STRtree` for spatial queries. When the viewport changes, `QueryShapesInBound()` only returns shapes that intersect the visible area. Without this, rendering 50,000 shapes would require checking every single one against the viewport bounds every frame.

Performance Impact

R-Tree query for 50,000 shapes: ~2ms. Linear scan for 50,000 shapes: ~150ms. At 60fps you have 16ms per frame. The R-Tree is what makes large datasets possible.

Chapter 2: What Is Missing and Why It Matters

Your rendering pipeline is solid. The gaps are in operational infrastructure: the plumbing that tells you what happened, when, and why when something goes wrong in the field. This chapter explains each gap specifically in the context of RePlot, not generic software.

2.1 Application Configuration

The Problem

Right now, performance thresholds and behavior settings are scattered as hardcoded constants across multiple files:

File	Constant	Current Value
OptimizedDeferredRenderer.cs	CACHE_THRESHOLD	1000
OptimizedDeferredRenderer.cs	LOD_THRESHOLD	20000
DrawingCanvasControl.cs	_maxShapesForSnapping	200
CanvasRenderer.cs	_majorGridSize	500
CanvasRenderer.cs	_minorGridSize	100
DrawingEngine.cs	MIN_ZOOM	0.0001
DrawingEngine.cs	MAX_ZOOM	50000.0
DrawingEngine.cs	ZOOM_STEP	1.4
SnapManager.cs	SnapPixelTolerance	15.0
UndoRedoManager.cs	maxUndoLevels	100

Why This Is a Problem for RePlot

A surveyor using RePlot in the field has a slow laptop with integrated graphics. They need to lower `CACHE_THRESHOLD` to 500 and increase `LOD_THRESHOLD`. Right now, that requires recompiling the application. With a configuration file, they just edit a JSON file.

The Solution: Typed Configuration

Create one strongly-typed class that holds ALL tunable settings. Load it once at startup from a JSON file. Inject it into every class that needs settings.

File: Infrastructure/Configuration/AppConfiguration.cs

```
public class AppConfiguration
{
    public CanvasSettings Canvas { get; set; } = new();
    public DatabaseSettings Database { get; set; } = new();
    public LoggingSettings Logging { get; set; } = new();
    public SnapSettings Snap { get; set; } = new();
}
```

```
public class CanvasSettings
{
    // From OptimizedDeferredRenderer.cs
    public int CacheThreshold { get; set; } = 1000;
    public int LodThreshold { get; set; } = 20000;

    // From DrawingEngine.cs
    public double MinZoom { get; set; } = 0.0001;
    public double MaxZoom { get; set; } = 50000.0;
    public double ZoomStep { get; set; } = 1.4;

    // From CanvasRenderer.cs
    public int MajorGridSize { get; set; } = 500;
    public int MinorGridSize { get; set; } = 100;
    public bool EnableAntiAlias { get; set; } = true;
    public string DefaultBackgroundColor { get; set; } = "#222933";

    // From UndoRedoManager
    public int MaxUndoLevels { get; set; } = 100;
}

public class SnapSettings
{
    // From SnapManager.cs
    public double SnapPixelTolerance { get; set; } = 15.0;
    // From DrawingCanvasControl.cs
    public int MaxShapesForSnapping { get; set; } = 200;
}

public class DatabaseSettings
{
    public string ConnectionString { get; set; } = "Data Source=replot.db";
}

public class LoggingSettings
{
    public string LogDirectory { get; set; } = "logs";
    public bool EnableDebugLogs { get; set; } = false;
    public int MaxLogFileSizeMB { get; set; } = 10;
}
```

File: Infrastructure/Configuration/ConfigurationLoader.cs

```
public static class ConfigurationLoader
{
    private const string ConfigFileName = "appsettings.json";

    public static AppConfiguration Load()
    {
        string configPath = Path.Combine(
```

```

        AppDomain.CurrentDomain.BaseDirectory, ConfigFileName));

    if (!File.Exists(configPath))
    {
        var defaults = new AppConfiguration();
        Save(defaults); // Create default config file
        return defaults;
    }

    string json = File.ReadAllText(configPath);
    return JsonSerializer.Deserialize<AppConfiguration>(json)
        ?? new AppConfiguration();
}

public static void Save(AppConfiguration config)
{
    string configPath = Path.Combine(
        AppDomain.CurrentDomain.BaseDirectory, ConfigFileName);
    var options = new JsonSerializerOptions { WriteIndented = true };
    File.WriteAllText(configPath,
        JsonSerializer.Serialize(config, options));
}
}

```

What Changes in Your Existing Code

The modifications are minimal. You replace hardcoded constants with values from the injected config object. Here are the exact changes:

Change 1: OptimizedDeferredRenderer.cs

BEFORE (hardcoded):

```

private const int CACHE_THRESHOLD = 1000;
private const int LOD_THRESHOLD = 20000;

```

AFTER (from config):

```

private readonly int _cacheThreshold;
private readonly int _lodThreshold;

public OptimizedDeferredRenderer(Size canvasSize,
    DrawingEngine engine, ShapeManager shapeManager,
    AppConfiguration config) // <-- ADD THIS PARAMETER
{
    _cacheThreshold = config.Canvas.CacheThreshold;
    _lodThreshold = config.Canvas.LodThreshold;
    // ... rest unchanged
}

```

Change 2: DrawingEngine.cs

BEFORE:

```
public const double MIN_ZOOM = 0.0001;
public const double MAX_ZOOM = 50000.0;
public const double ZOOM_STEP = 1.4;
```

AFTER:

```
public readonly double MinZoom;
public readonly double MaxZoom;
public readonly double ZoomStep;

public DrawingEngine(Size canvasSize, AppConfig configuration config)
{
    _canvasSize = canvasSize;
    MinZoom = config.Canvas.MinZoom;
    MaxZoom = config.Canvas.MaxZoom;
    ZoomStep = config.Canvas.ZoomStep;
}
```

Change 3: DrawingCanvasControl.cs

BEFORE (in constructor or field):

```
private int _maxShapesForSnapping = 200;
```

AFTER:

```
private readonly int _maxShapesForSnapping;

// In constructor:
_maxShapesForSnapping = config.Snap.MaxShapesForSnapping;
```

Chapter 3: Logging Infrastructure

3.1 Why Logging Matters for RePlot Specifically

When a surveyor imports 10,000 parcel boundaries from a DXF file and 47 shapes are silently dropped, how do you know which ones and why? When the canvas freezes for 3 seconds during a zoom operation, how do you reproduce it? When a parcel split creates an invalid polygon, how do you see the before and after coordinates?

Without logging, the answer to all these questions is: you do not know. You have zero diagnostic capability in production.

3.2 What to Install

Add these NuGet packages to your .csproj:

Serilog	(core library)
Serilog.Sinks.File	(rolling daily log files)
Serilog.Sinks.Debug	(Visual Studio Output window)

Serilog is the industry standard for .NET desktop applications. It is used by Microsoft's own tools internally.

3.3 Implementation

File: Infrastructure/Logging/IAppLogger.cs

```
public interface IAppLogger
{
    void Debug(string message, params object[] args);
    void Info(string message, params object[] args);
    void Warn(string message, params object[] args);
    void Error(string message, Exception? ex = null);
    void Error(string message, params object[] args);
    IDisposable BeginTimedOperation(string operationName);
}
```

The interface exists so your code depends on IAppLogger, not on Serilog directly. If you ever switch to NLog or another framework, you change one file.

File: Infrastructure/Logging/SerilogAppLogger.cs

```
public class SerilogAppLogger : IAppLogger
{
    private readonly ILogger _logger;

    public SerilogAppLogger(AppConfiguration config)
    {
        string logDir = Path.Combine(
            Environment.GetFolderPath(
```



```

        Environment.SpecialFolder.ApplicationData),
        "RePlot", config.Logging.LogDirectory);

_logger = new LoggerConfiguration()
    .MinimumLevel.Is(config.Logging.EnableDebugLogs
        ? LogEventLevel.Debug
        : LogEventLevel.Information)
    .WriteTo.Debug() // VS Output window
    .WriteTo.File(
        Path.Combine(logDir, "replot-.log"),
        rollingInterval: RollingInterval.Day,
        fileSizeLimitBytes: config.Logging.MaxLogFileSizeMB
            * 1024 * 1024,
        retainedFileCountLimit: 30)
    .CreateLogger();
}

public void Debug(string msg, params object[] args)
    => _logger.Debug(msg, args);

public void Info(string msg, params object[] args)
    => _logger.Information(msg, args);

public void Warn(string msg, params object[] args)
    => _logger.Warning(msg, args);

public void Error(string msg, Exception? ex = null)
    => _logger.Error(ex, msg);

public void Error(string msg, params object[] args)
    => _logger.Error(msg, args);

public IDisposable BeginTimedOperation(string name)
    => new TimedOperation(_logger, name);
}

```

Timed Operation Helper (nested class)

```

private class TimedOperation : IDisposable
{
    private readonly ILogger _logger;
    private readonly string _name;
    private readonly Stopwatch _sw;

    public TimedOperation(ILogger logger, string name)
    {
        _logger = logger;
        _name = name;
        _sw = Stopwatch.StartNew();
        _logger.Information("{Operation} started", name);
    }
}

```

```

    }

    public void Dispose()
    {
        _sw.Stop();
        _logger.Information("{Operation} completed in {Elapsed}ms",
            _name, _sw.ElapsedMilliseconds);
    }
}

```

3.4 Where to Add Log Calls in Your Existing Code

You do not need to log everything. These are the critical points specific to RePlot:

Class	What to Log	Example Call
DrawingEngine	Every zoom with scale value	<code>_logger.Debug("Zoom: {Scale}", _zoomScale)</code>
ShapeManager	Add/remove count + elapsed	<code>_logger.Info("BulkAdd {Count} shapes", count)</code>
DXF Import	Every row, every failure	<code>_logger.Warn("Row {N} skipped: {Reason}")</code>
Database	Every transaction	<code>_logger.Info("Saved {Count} parcels")</code>
Canvas Paint	Render time (debug only)	<code>_logger.Debug("Frame: {Ms}ms", elapsed)</code>
RTreeSpatialIndex	Rebuild time + shape count	<code>_logger.Debug("Rebuild: {Count} in {Ms}ms")</code>
Undo/Redo	Command executed/undone	<code>_logger.Info("Undo: {Desc}", cmd.Description)</code>

Practical Example: Adding Logging to ShapeManager.BulkAddShapes

BEFORE:

```

public void BulkAddShapes(IEnumerable<IShape> shapes)
{
    var shapeList = shapes as IList<IShape> ?? shapes.ToList();
    _shapes.AddRange(shapeList);
    RebuildSpatialIndex();
}

```

AFTER:

```

public void BulkAddShapes(IEnumerable<IShape> shapes)
{
    var shapeList = shapes as IList<IShape> ?? shapes.ToList();
    using (_logger.BeginTimedOperation("BulkAddShapes"))
    {

```

```
        _shapes.AddRange(shapeList);  
        RebuildSpatialIndex();  
    }  
    _logger.Info("Added {Count} shapes. Total: {Total}",  
        shapeList.Count, _shapes.Count);  
}
```

Chapter 4: Dependency Injection

4.1 What DI Solves in Your Code

Right now, `DrawingCanvasControl` manually creates all its dependencies. Look at what is happening conceptually in your initialization:

```
// Inside DrawingCanvasControl (current pattern)
_engine = new DrawingEngine(canvasSize);           // hardcoded
_shapeManager = new ShapeManager(worldBounds);     // hardcoded
_renderer = new CanvasRenderer();                  // hardcoded
_undoManager = new UndoRedoManager();               // hardcoded
_deferredRenderer = new OptimizedDeferredRenderer(
    canvasSize, _engine, _shapeManager);            // hardcoded
```

This works, but it creates three problems:

- You cannot swap implementations without editing `DrawingCanvasControl`. Want to test with a mock `ShapeManager` that always returns 5 shapes? You cannot.
- Every component that needs logging or configuration must somehow get those objects. Without DI, you end up passing them through 4 layers of constructors manually.
- Components are tightly coupled. `DrawingCanvasControl` knows the concrete types of everything it uses.

4.2 How to Wire DI in WinForms (.NET 10)

You are using .NET 10 which includes `Microsoft.Extensions.DependencyInjection` in the SDK. No additional NuGet package needed for the DI container itself.

File: `Infrastructure/DI/ServiceRegistration.cs`

```
public static class ServiceRegistration
{
    public static IServiceProvider Configure()
    {
        var services = new ServiceCollection();

        // Infrastructure (one instance for entire app)
        var config = ConfigurationLoader.Load();
        services.AddSingleton(config);
        services.AddSingleton<IAppLogger, SerilogAppLogger>();

        // Canvas components (one instance per app)
        services.AddSingleton<DrawingEngine>();
        services.AddSingleton<ShapeManager>();
        services.AddSingleton<CanvasRenderer>();
        services.AddSingleton<UndoRedoManager>();
        services.AddSingleton<SnapManager>();
    }
}
```

```

        // Forms
        services.AddTransient<frmMain>();

        return services.BuildServiceProvider();
    }
}

```

File: Program.cs (Modified)

BEFORE:

```

static void Main()
{
    // ... DPI, encoding setup ...
    Application.Run(new frmMain());
}

```

AFTER:

```

static void Main()
{
    // ... DPI, encoding setup (keep all existing) ...

    var provider = ServiceRegistration.Configure();
    Application.Run(provider.GetRequiredService<frmMain>());
}

```

What Changes in DrawingCanvasControl

BEFORE (manual construction):

```

public DrawingCanvasControl()
{
    InitializeComponent();
    _engine = new DrawingEngine(panelCanvas.Size);
    _shapeManager = new ShapeManager(worldBounds);
    _renderer = new CanvasRenderer();
    _undoManager = new UndoRedoManager();
}

```

AFTER (injected dependencies):

```

public DrawingCanvasControl(
    DrawingEngine engine,
    ShapeManager shapeManager,
    CanvasRenderer renderer,
    UndoRedoManager undoManager,
    IAppLogger logger,
    AppConfig configuration)
{
    InitializeComponent();
}

```

```
_engine = engine;
_shapeManager = shapeManager;
_renderer = renderer;
_undoManager = undoManager;
_logger = logger;
_config = config;
_maxShapesForSnapping = config.Snap.MaxShapesForSnapping;
}
```

Key Insight

The behavior of DrawingCanvasControl does not change at all. The only change is WHERE the objects come from. Instead of creating them itself, it receives them from outside. This is the entire concept of dependency injection.

Chapter 5: Operation Pipeline (Middleware)

5.1 The Problem This Solves for Land Replotting

When you split a parcel in RePlot, the following things must happen in order:

4. Validate the split line does not exit the parcel boundary
5. Execute the split geometry (create two new polygons)
6. Recalculate areas of both resulting parcels
7. Validate new areas against minimum plot size rules
8. Register the operation to the undo stack
9. Log the operation with before/after state
10. Trigger UI refresh

Right now, if you implemented this, all seven steps would live inside a single event handler method in `DrawingCanvasControl`. When you later add parcel merge, parcel move, and boundary adjustment, you would duplicate the validation, undo registration, logging, and area recalculation code in each one.

The pipeline pattern means each concern lives in exactly one place and runs automatically for every parcel operation.

5.2 The Pipeline Architecture

File: `Core/Pipeline/IOperationMiddleware.cs`

```
public interface IOperationMiddleware
{
    Task<OperationResult> ExecuteAsync(
        ParcelOperation operation,
        Func<Task<OperationResult>> next);
}
```

Each middleware receives the operation and a delegate to call the next middleware. It can run code before, after, or instead of calling `next()`.

File: `Core/Pipeline/ParcelOperation.cs`

```
public class ParcelOperation
{
    public OperationType Type { get; set; } // Split, Merge, Move, Adjust
    public IShape TargetParcel { get; set; }
    public object Parameters { get; set; } // Split line, merge target, etc.
    public Dictionary<string, object> Metadata { get; } = new();
}
```

```

public class OperationResult
{
    public bool Success { get; set; }
    public string Message { get; set; }
    public List<IShape> AffectedShapes { get; set; } = new();
}

public enum OperationType
{ Split, Merge, Move, BoundaryAdjust, AreaRecalculate }

```

File: Core/Pipeline/OperationPipeline.cs

```

public class OperationPipeline
{
    private readonly List<IOperationMiddleware> _middleware = new();

    public OperationPipeline Use(IOperationMiddleware mw)
    {
        _middleware.Add(mw);
        return this; // fluent API
    }

    public async Task<OperationResult> ExecuteAsync(
        ParcelOperation operation)
    {
        // Build the chain from inside out
        Func<Task<OperationResult>> chain =
            () => Task.FromResult(new OperationResult
                { Success = true, Message = "Completed" });

        for (int i = _middleware.Count - 1; i >= 0; i--)
        {
            var mw = _middleware[i];
            var next = chain;
            chain = () => mw.ExecuteAsync(operation, next);
        }

        return await chain();
    }
}

```

5.3 Concrete Middleware Examples

ValidationMiddleware.cs

```

public class ValidationMiddleware : IOperationMiddleware
{
    public async Task<OperationResult> ExecuteAsync(
        ParcelOperation op, Func<Task<OperationResult>> next)
    {

```



```

        // PRE-OPERATION: Validate before executing
        if (op.TargetParcel == null)
            return new OperationResult
                { Success = false, Message = "No parcel selected" };

        if (op.Type == OperationType.Split)
        {
            var splitLine = op.Parameters as LineShape;
            if (!SplitLineIntersectsParcel(
                op.TargetParcel, splitLine))
                return new OperationResult { Success = false,
                    Message = "Split line does not cross parcel" };
        }

        // Validation passed - continue to next middleware
        return await next();
    }
}

```

LoggingMiddleware.cs

```

public class LoggingMiddleware : IOperationMiddleware
{
    private readonly IAppLogger _logger;

    public LoggingMiddleware(IAppLogger logger)
        => _logger = logger;

    public async Task<OperationResult> ExecuteAsync(
        ParcelOperation op, Func<Task<OperationResult>> next)
    {
        _logger.Info("{OpType} started on parcel {Id}",
            op.Type, op.TargetParcel?.Id);
        var sw = Stopwatch.StartNew();

        var result = await next();

        sw.Stop();
        if (result.Success)
            _logger.Info("{OpType} completed in {Ms}ms. " +
                "Affected: {Count} shapes",
                op.Type, sw.ElapsedMilliseconds,
                result.AffectedShapes.Count);
        else
            _logger.Warn("{OpType} FAILED: {Msg}",
                op.Type, result.Message);

        return result;
    }
}

```

UndoRegistrationMiddleware.cs

```
public class UndoRegistrationMiddleware : IOperationMiddleware
{
    private readonly UndoRedoManager _undoManager;

    public UndoRegistrationMiddleware(UndoRedoManager undoManager)
        => _undoManager = undoManager;

    public async Task<OperationResult> ExecuteAsync(
        ParcelOperation op, Func<Task<OperationResult>> next)
    {
        // Capture state BEFORE operation
        var beforeState = op.TargetParcel?.Clone();

        var result = await next();

        if (result.Success && beforeState != null)
        {
            // Register undo command with before/after state
            _undoManager.ExecuteCommand(
                new ParcelOperationCommand(
                    op.Type, beforeState,
                    result.AffectedShapes));
        }

        return result;
    }
}
```

AreaRecalculationMiddleware.cs

```
public class AreaRecalculationMiddleware : IOperationMiddleware
{
    public async Task<OperationResult> ExecuteAsync(
        ParcelOperation op, Func<Task<OperationResult>> next)
    {
        var result = await next();

        // POST-OPERATION: Recalculate areas
        if (result.Success)
        {
            foreach (var shape in result.AffectedShapes)
            {
                if (shape is PolygonShape polygon)
                {
                    double area = CalculateArea(polygon);
                    shape.Properties["Area"] = area;
                    shape.Properties["AreaSqM"] = area;
                }
            }
        }
    }
}
```

```

        }
    }
    }
    return result;
}
}

```

5.4 How to Wire the Pipeline

```

// In ServiceRegistration.cs or wherever you build the pipeline:
var pipeline = new OperationPipeline()
    .Use(new LoggingMiddleware(logger))
    .Use(new ValidationMiddleware())
    .Use(new UndoRegistrationMiddleware(undoManager))
    .Use(new AreaRecalculationMiddleware());

```

Order matters. Logging wraps everything (measures total time). Validation runs before execution. Undo captures state before and registers after. Area recalculation runs after geometry changes.

5.5 How to Call the Pipeline From Your Canvas

```

// Instead of scattered logic in an event handler:
private async void ExecuteParcelSplit(IShape parcel, LineShape splitLine)
{
    var op = new ParcelOperation
    {
        Type = OperationType.Split,
        TargetParcel = parcel,
        Parameters = splitLine
    };

    var result = await _pipeline.ExecuteAsync(op);

    if (result.Success)
    {
        _deferredRenderer.RenderNow();
        panelCanvas.Invalidate();
    }
    else
    {
        MessageBox.Show(result.Message, "Operation Failed");
    }
}
}

```

Chapter 6: Critical Fix - R-Tree Rebuild on Every Insert

Performance Bug

Your `RTreeSpatialIndex.Insert()` method calls `Rebuild()` after every single shape insertion. When you import 10,000 shapes via `BulkAddShapes`, this rebuilds the entire R-Tree 10,000 times. This is an $O(n^2)$ operation that takes minutes instead of milliseconds.

Current Code (RTreeSpatialIndex.cs)

```
public void Insert(IShape shape)
{
    _allShapes.Add(shape);
    Rebuild(); // PROBLEM: Full rebuild on every insert!
}
```

The Fix: Dirty Flag Pattern

```
public class RTreeSpatialIndex
{
    private STRtree<IShape> _index;
    private List<IShape> _allShapes = new();
    private bool _dirty = true;

    public void Insert(IShape shape)
    {
        _allShapes.Add(shape);
        _dirty = true; // Mark dirty, don't rebuild
    }

    public bool Remove(IShape shape)
    {
        bool removed = _allShapes.Remove(shape);
        if (removed) _dirty = true;
        return removed;
    }

    public List<IShape> Query(RectangleD area)
    {
        if (_dirty) Rebuild(); // Rebuild only when actually queried
        // ... existing query logic ...
    }

    public void Rebuild()
    {
        _index = new STRtree<IShape>();
        foreach (var shape in _allShapes)
            _index.Insert(ToEnvelope(shape.GetBoundingBox()), shape);
        _dirty = false;
    }
}
```

```
| }
```

Impact

Importing 10,000 shapes: Before fix: ~45 seconds (10,000 rebuilds). After fix: ~50 milliseconds (1 rebuild when first query happens). This is a 900x improvement.

Chapter 7: Remove Redundant Code

7.1 SimpleDeferredRenderer - Delete It

You have two deferred renderers: SimpleDeferredRenderer and OptimizedDeferredRenderer. SimpleDeferredRenderer is a subset of OptimizedDeferredRenderer with less functionality (no pan buffer, no LOD). It is never used in your DrawingCanvasControl. Delete the file.

Files to delete: Rendering/SimpleDeferredRenderer.cs

7.2 Duplicate Viewport Methods in DrawingEngine

DrawingEngine has both GetViewportBounds() and GetVisibleWorldRectangle() which do the same thing with slightly different logic. Keep GetViewportBounds() (the more robust one that checks all four corners) and remove GetVisibleWorldRectangle(). Update the two call sites in DrawingCanvasControl that use GetVisibleWorldRectangle to call GetViewportBounds instead.

7.3 Unused Render() Method in CanvasRenderer

CanvasRenderer has a full Render() method that renders grid + shapes + overlays. But your panelCanvas_Paint handler calls each rendering step individually (RenderGrid, DrawAxisMarkers, DrawShapesCache, etc.). The monolithic Render() method is dead code. Remove it to avoid confusion about which render path is actually used.

Chapter 8: Implementation Order (Step by Step)

Do these in order. Each step builds on the previous one. Do not skip ahead.

Phase 1: Fix Bugs First (30 minutes)

11. Fix `RTreeSpatialIndex.Insert()` to use dirty flag pattern (Chapter 6)
12. Delete `SimpleDeferredRenderer.cs`
13. Remove `GetVisibleWorldRectangle()` from `DrawingEngine`, update call sites
14. Remove dead `Render()` method from `CanvasRenderer`
15. Build and verify no compile errors

Phase 2: Configuration (1 hour)

16. Create `Infrastructure/Configuration/` folder
17. Create `AppConfiguration.cs` with all settings classes
18. Create `ConfigurationLoader.cs`
19. Add `using System.Text.Json` to both files
20. Replace hardcoded constants in `OptimizedDeferredRenderer`, `DrawingEngine`, `SnapManager`, `DrawingCanvasControl`
21. Build and verify behavior is identical

Phase 3: Logging (1.5 hours)

22. Add Serilog NuGet packages to `.csproj`
23. Create `Infrastructure/Logging/` folder
24. Create `IAppLogger.cs` interface
25. Create `SerilogAppLogger.cs` implementation
26. Add `_logger` parameter to `ShapeManager`, `DrawingEngine` constructors
27. Add log calls to the 7 critical points listed in Chapter 3
28. Build, run, verify logs appear in `%AppData%/RePlot/logs/`

Phase 4: Dependency Injection (1 hour)

29. Create `Infrastructure/DI/ServiceRegistration.cs`
30. Modify `Program.cs` to use DI container
31. Add constructor injection to `DrawingCanvasControl`
32. Add constructor injection to `frmMain`

33. Build and verify everything still works identically

Phase 5: Operation Pipeline (2 hours)

- 34. Create Core/Pipeline/ folder
- 35. Create IOperationMiddleware.cs, ParcelOperation.cs, OperationResult.cs
- 36. Create OperationPipeline.cs
- 37. Create LoggingMiddleware.cs (simplest - start here)
- 38. Create ValidationMiddleware.cs
- 39. Create UndoRegistrationMiddleware.cs
- 40. Wire pipeline in ServiceRegistration.cs
- 41. Use pipeline for one operation (e.g. add shape) as proof of concept

Chapter 9: Target Folder Structure

```

RePlot/
  Infrastructure/
    Configuration/
      AppConfiguraton.cs
      ConfigurationLoader.cs
    Logging/
      IAppLogger.cs
      SerilogAppLogger.cs
    DI/
      ServiceRegistration.cs

  Core/
    Pipeline/
      IOperationMiddleware.cs
      OperationPipeline.cs
      ParcelOperation.cs
      OperationResult.cs
    Middleware/
      ValidationMiddleware.cs
      LoggingMiddleware.cs
      UndoRegistrationMiddleware.cs
      AreaRecalculationMiddleware.cs

  MapCanvas/                                (existing, keep structure)
    Core/
      DrawingEngine.cs                      (modified: accept config)
      ShapeManager.cs                      (modified: accept logger)
      SpatialIndex/
        RTreeSpatialIndex.cs              (modified: dirty flag)
      Commands/
        ICommand.cs                      (unchanged)
        UndoRedoManager.cs               (modified: accept config)
        ShapeCommands.cs                 (unchanged)
    Rendering/
      CanvasRenderer.cs                   (modified: accept config)
      OptimizedDeferredRenderer.cs        (modified: accept config)
      [DELETED: SimpleDeferredRenderer.cs]
    Models/
      Shapes/                             (unchanged)
      Snapping/                           (unchanged)
    Data/
      ShapeRepository.cs                  (unchanged, implement later)

  CustomControls/
    DrawingCanvasControl.cs               (modified: accept injected deps)

  Services/                               (existing, unchanged)
  Forms/                                  (existing, frmMain modified)
  Repositories/                           (existing, unchanged)

```

Models/	(existing, unchanged)
Program.cs	(modified: wire DI)
appsettings.json	(NEW: auto-generated on first run)

Chapter 10: Canvas Rendering Quality at Extreme Zoom

You mentioned wanting no graphics issues during zooming, panning, and at high zoom levels. Here are the specific techniques that address this, mapped to your existing code.

10.1 Sub-Pixel Rendering at High Zoom

At very high zoom levels (above 1000x), GDI+ starts to show floating-point precision artifacts. Lines may appear jagged or misaligned by 1 pixel. Your code already handles this partially with `PixelFormatMode.HighQuality` in `CanvasRenderer.Render()`. But there is one additional fix:

```
// In CanvasRenderer, when drawing shapes at high zoom:
if (engine.ZoomScale > 500)
{
    g.PixelOffsetMode = PixelOffsetMode.Half;
    // Half-pixel offset prevents thin line disappearance
}
```

10.2 Grid Rendering at Extreme Zoom

Your adaptive grid system already handles this well with hysteresis (`MIN_MINOR_PIXELS` and `MAX_MINOR_PIXELS` bands). The one improvement is to add a maximum grid line count cap to prevent slowdown at certain zoom levels where many lines are visible:

```
// Add to RenderGrid(), after calculating grid bounds:
int estimatedLines = (int)((worldRight - worldLeft) / adaptiveMinorSize)
    + (int)((worldTop - worldBottom) / adaptiveMinorSize);
if (estimatedLines > 2000)
    return; // Skip grid entirely - too dense to be useful
```

10.3 Pan Buffer Tearing

During fast panning, the pan buffer shift can cause a visible tear at the canvas edges where the old buffer ends and the background shows. Your current code handles this by letting the background color show through. A refinement is to slightly over-render the cache:

```
// In OptimizedDeferredRenderer, when creating cache bitmap:
int padding = 100; // Extra pixels around viewport
_shapesCache = new Bitmap(
    _canvasSize.Width + padding * 2,
    _canvasSize.Height + padding * 2);
```

This gives you 100 pixels of buffer on each edge before the background is exposed during pan. The overhead is negligible (a few KB of extra bitmap).

10.4 Double Buffering

Ensure your `panelCanvas` has double buffering enabled. In the `DrawingCanvasControl` constructor:

```
panelCanvas.GetType().GetProperty("DoubleBuffered",  
    BindingFlags.Instance | BindingFlags.NonPublic)  
?.SetValue(panelCanvas, true);
```

This prevents flicker during Paint events by rendering to an off-screen buffer first, then blitting to screen in one operation.

Summary: What Changes vs What Stays

Component	Status	Change Type
DrawingEngine.cs	Modified	Accept config instead of consts
CanvasRenderer.cs	Modified	Accept config, remove dead Render()
OptimizedDeferredRenderer.cs	Modified	Accept config instead of consts
SimpleDeferredRenderer.cs	DELETED	Redundant with Optimized
ShapeManager.cs	Modified	Accept logger, use timed ops
RTreeSpatialIndex.cs	Modified	Dirty flag pattern (bug fix)
DrawingCanvasControl.cs	Modified	Constructor injection
SnapManager.cs	Modified	Accept config for tolerance
UndoRedoManager.cs	Modified	Accept config for max levels
Program.cs	Modified	Wire DI container
IShape.cs	Unchanged	No changes needed
ShapeCommands.cs	Unchanged	No changes needed
ICommand.cs	Unchanged	No changes needed
All shape classes	Unchanged	No changes needed
All Forms	Unchanged	Except frmMain constructor
All Services	Unchanged	No changes needed
Database layer	Unchanged	No changes needed

Total Effort

Phase 1 (bug fixes): 30 min. Phase 2 (config): 1 hour. Phase 3 (logging): 1.5 hours. Phase 4 (DI): 1 hour. Phase 5 (pipeline): 2 hours. Total: approximately 6 hours of focused work.

Your canvas rendering architecture is already solid. These changes add the operational infrastructure that turns a working prototype into a production application that can be diagnosed, configured, and maintained when deployed in the field.